Ctrl + S

homework 9, version 3

Homework 9: *Epidemic modeling III*

`18.S191`, Fall 2023

This notebook contains *built-in, live answer checks*! In some exercises you will see a coloured box, which runs a test case on your code, and provides feedback based on the result. Simply edit the code, run it, and the check runs again.

Feel free to ask questions!

Let's create a package environment:

```
using Plots, PlutoUI, LinearAlgebra
```

In this problem set, we will look at a simple **spatial** agent-based epidemic model: agents can interact only with other agents that are *nearby*. (In Homework 7, any agent could interact with any other, which is not realistic.)

A simple approach is to use **discrete space**: each agent lives in one cell of a square grid. For simplicity we will allow no more than one agent in each cell, but this requires some care to design the rules of the model to respect this.

We will adapt some functionality from Homework 7. You should copy and paste your code from that homework into this notebook.

Exercise 1: *Wandering at random in 2D*

In this exercise we will implement a **random walk** on a 2D lattice (grid). At each time step, a walker jumps to a neighbouring position at random (i.e. chosen with uniform probability from the available adjacent positions).

Exercise 1.1

We define a struct type `Coordinate` that contains integers `x` and `y`.

```
struct Coordinate
    x::Int
    y::Int
end
```

👉 Construct a `Coordinate` located at the origin.

```
origin =  Coordinate(0, 0)
```

```
origin = Coordinate(0,0)
```

Got it!

Yay ❤

👉 Write a function `make_tuple` that takes an object of type `Coordinate` and returns the corresponding tuple `(x, y)`. Boring, but useful later!

```
make_tuple (generic function with 1 method)
```

```
function make_tuple(c::Coordinate)
    a=c.x
    b=c.y
    return (a,b)
end
```

Got it!

You got the right answer!

```
if !@isdefined(make_tuple)
    not_defined(:make_tuple)
else
    let
        result = make_tuple(Coordinate(2,1))
        if result isa Missing
            still_missing()
        elseif !(result isa Tuple)
            keep_working(md"Make sure that you return a `Tuple`, like so: `return (1, 2)`.")
        else
            if result == (2, 1)
                correct()
            else
                keep_working()
            end
        end
    end
end
```

Exercise 1.2

In Julia, operations like `+` and `*` are just functions, and they are treated like any other function in the language. The only special property you can use the *infix notation*: you can write

```
1 + 2
```

instead of

```
+(1, 2)
```

(There are lots of special 'infixable' function names that you can use for your own functions!)

When you call it with the prefix notation, it becomes clear that it really is 'just another function', with lots of predefined methods.

```
3
```

```
1 +(1, 2)
```

```
+ (generic function with 243 methods)
```

```
1 +
```

Extending + in the wild

Because it is a function, we can add our own methods to it! This feature is super useful in general languages like Julia and Python, because it lets you use familiar syntax (`a + b*c`) on objects that are not necessarily numbers!

One example we've see before is the `RGB` type in the first lectures. You are able to do:

```
0.5 * RGB(0.1, 0.7, 0.6)
```

to multiply each color channel by 0.5. This is possible because `Images.jl` wrote a method:

```
*(::Real, ::AbstractRGB)::AbstractRGB
```

👉 Implement addition on two `Coordinate` structs by adding a method to `Base.:+`

```
1 function Base.:+(a::Coordinate, b::Coordinate)
2     return Coordinate(a.x+b.x,a.y+b.y)
3 end
```

```
Coordinate(13, 14)
```

```
1 Coordinate(3,4) + Coordinate(10,10) # uncomment to check + works
```

Pluto has some trouble here, you need to manually re-run the cell above!

Got it!

Keep it up!

Exercise 1.3

In our model, agents will be able to walk in 4 directions: up, down, left and right. We can define these directions as `Coordinate`s.

```
possible_moves = [Coordinate(1, 0), Coordinate(0, 1), Coordinate(-1, 0), Coordinate(0, -1)]
```

```
possible_moves = [
    Coordinate( 1, 0),
    Coordinate( 0, 1),
    Coordinate(-1, 0),
    Coordinate( 0,-1),
]
```

👉 `rand(possible_moves)` gives a random possible move. Add this to the coordinate `Coordinate(4,5)` and see that it moves to a valid neighbor.

```
Coordinate(3, 5)
```

```
Coordinate(4,5)+ rand(possible_moves)
```

We are able to make a `Coordinate` perform one random step, by adding a move to it. Great!

👉 Write a function `trajectory` that calculates a trajectory of a `Wanderer w` when performing `n` steps., i.e. the sequence of positions that the walker finds itself in.

Possible steps:

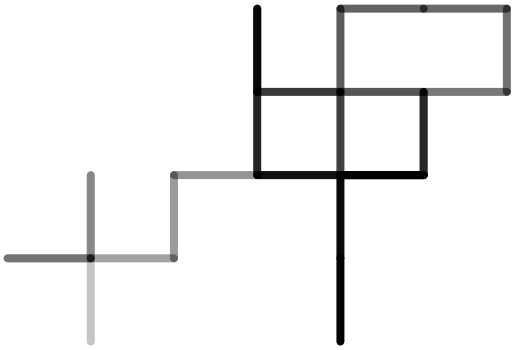
- Use `rand(possible_moves, n)` to generate a vector of `n` random moves. Each possible move will be equally likely.
- To compute the trajectory you can use either of the following two approaches:
    1. 🆒 Use the function `accumulate` (see the live docs for `accumulate`). Use `+` as the function passed to `accumulate` and the `w` as the starting value (`init` keyword argument).
    2. Use a `for` loop calling `+`.

```
trajectory (generic function with 1 method)
```

```
function trajectory(w::Coordinate, n::Int)
    moves=rand(possible_moves,n)

    return accumulate(+,moves,init=w)
end
```

```
test_trajectory = [Coordinate(4, 3), Coordinate(4, 4), Coordinate(4, 5), Coordinate(5, 5), Coordinate(4, 5), Coord
```

```
test_trajectory = trajectory(Coordinate(4,4), 30) # uncomment to test
```

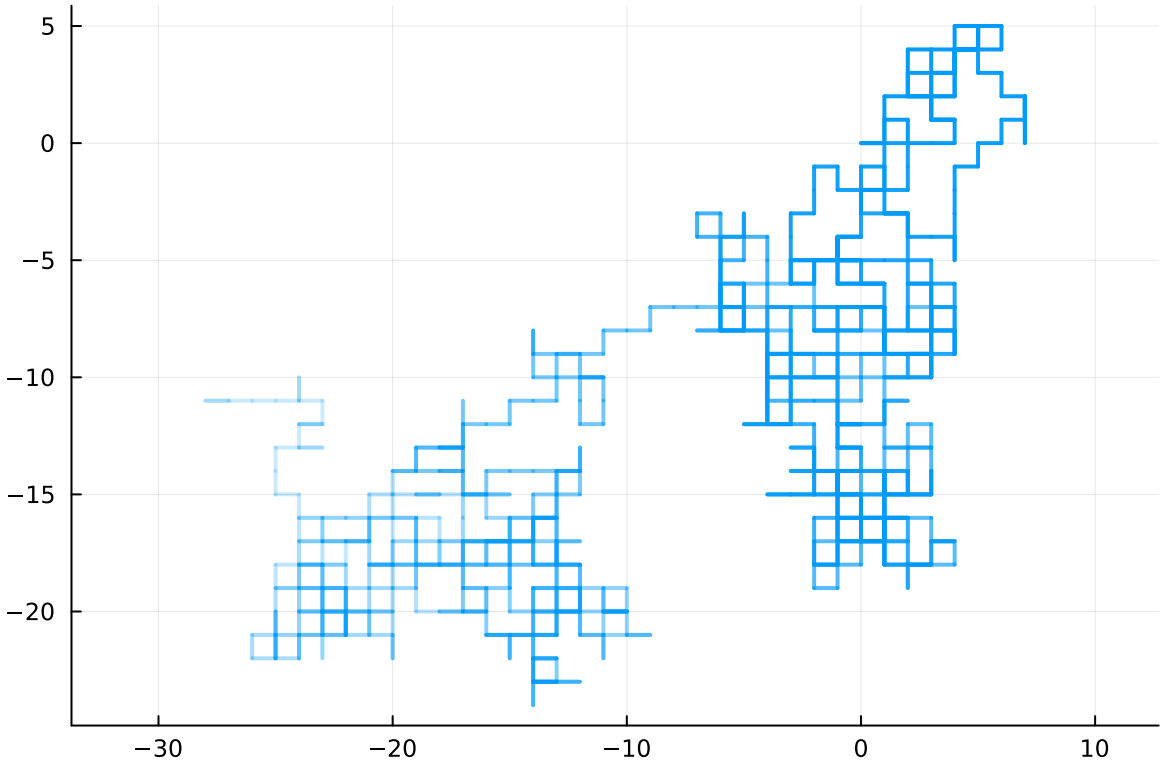


Got it!

Great!

```
plot_trajectory! (generic function with 1 method)
```

```
function plot_trajectory!(p::Plots.Plot, trajectory::Vector; kwargs...)
    plot!(p, make_tuple.(trajectory);
        label=nothing,
        linewidth=2,
        linealpha=LinRange(1.0, 0.2, length(trajectory)),
        kwargs...)
end
```



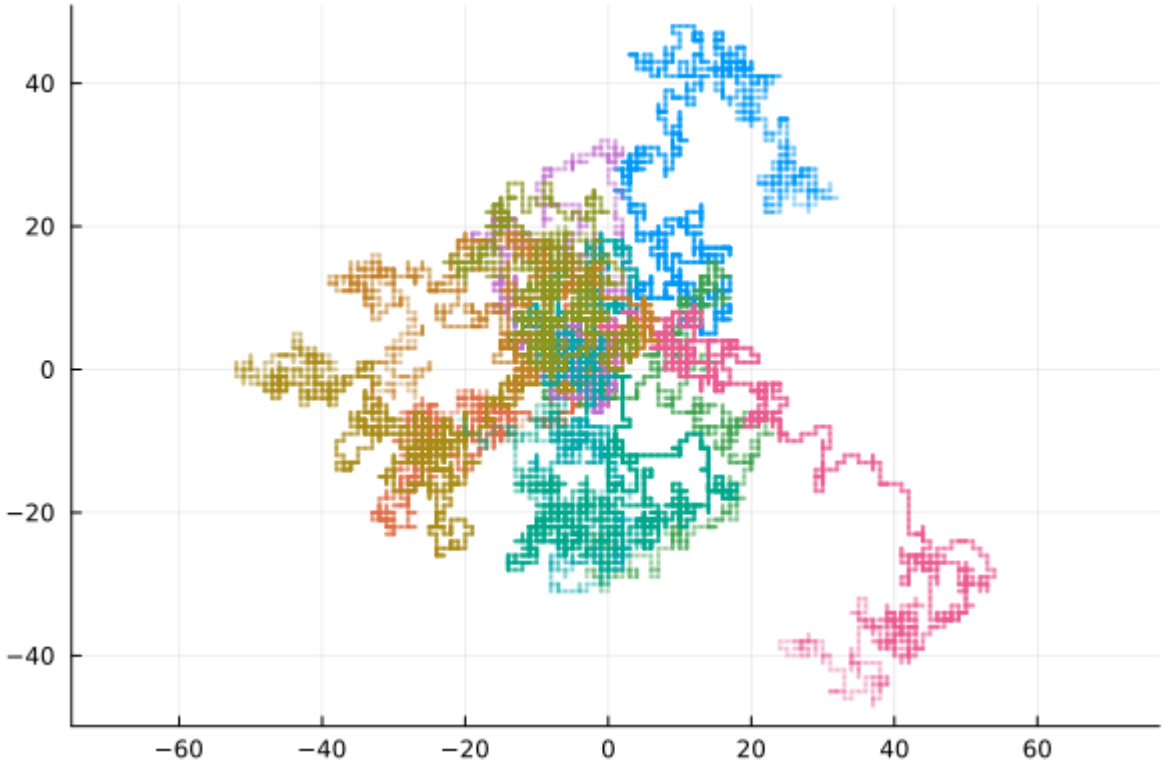

```
let
        long_trajectory = trajectory(Coordinate(4,4), 1000)

        p = plot(ratio=1)
        plot_trajectory!(p, long_trajectory)
        p
end
```

Exercise 1.4

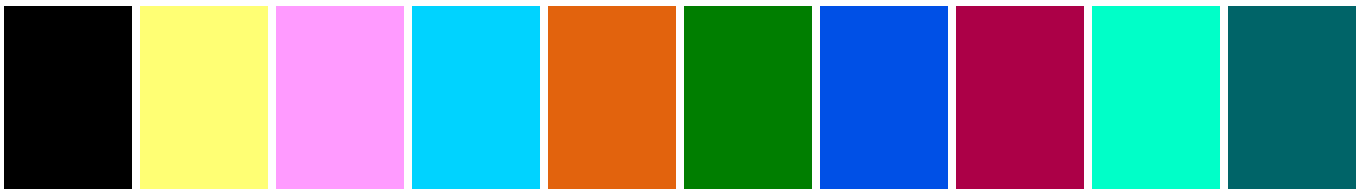
👉 Plot 10 trajectories of length 1000 on a single figure, all starting at the origin. Use the function `plot_trajectory!` as demonstrated above.

Remember from last week that you can compose plots like this:

```
let
    # Create a new plot with aspect ratio 1:1
    p = plot(ratio=1)

    plot_trajectory!(p, test_trajectory)      # plot one trajectory
    plot_trajectory!(p, another_trajectory)   # plot the second one
    ...

    p
end
```




```
let
    p = plot(ratio=1)
    for i in 1:10
        long_trajectory = trajectory(origin, 1000)
        plot_trajectory!(p, long_trajectory)
    end
        p
end
```



```
distinguishable_colors(10)
```

Exercise 1.5

Agents live in a box of side length $2L$, centered at the origin. We need to decide (i.e. model) what happens when they reach the walls of the box (boundaries), in other words what kind of **boundary conditions** to use.

One relatively simple boundary condition is a **collision boundary**:

> Each wall of the box is a wall, modelled using "collision": if the walker tries to jump beyond the wall, it ends up at the position inside the box that is closest to the goal.

👉 Write a function `collide_boundary` which takes a `Coordinate c` and a size L, and returns a new coordinate that lies inside the box (i.e. $[-L, L] \times [-L, L]$), but is closest to `c`. This is similar to `extend_mat` from Homework 1.

```
collide_boundary (generic function with 1 method)
```

```
function collide_boundary(c::Coordinate, L::Number)
    x=c.x
    y=c.y
    x_new=clamp(x,-L,L)
    y_new=clamp(y,-L,L)
    return Coordinate(x_new,y_new)
end
```

```
Coordinate(10, 4)
```

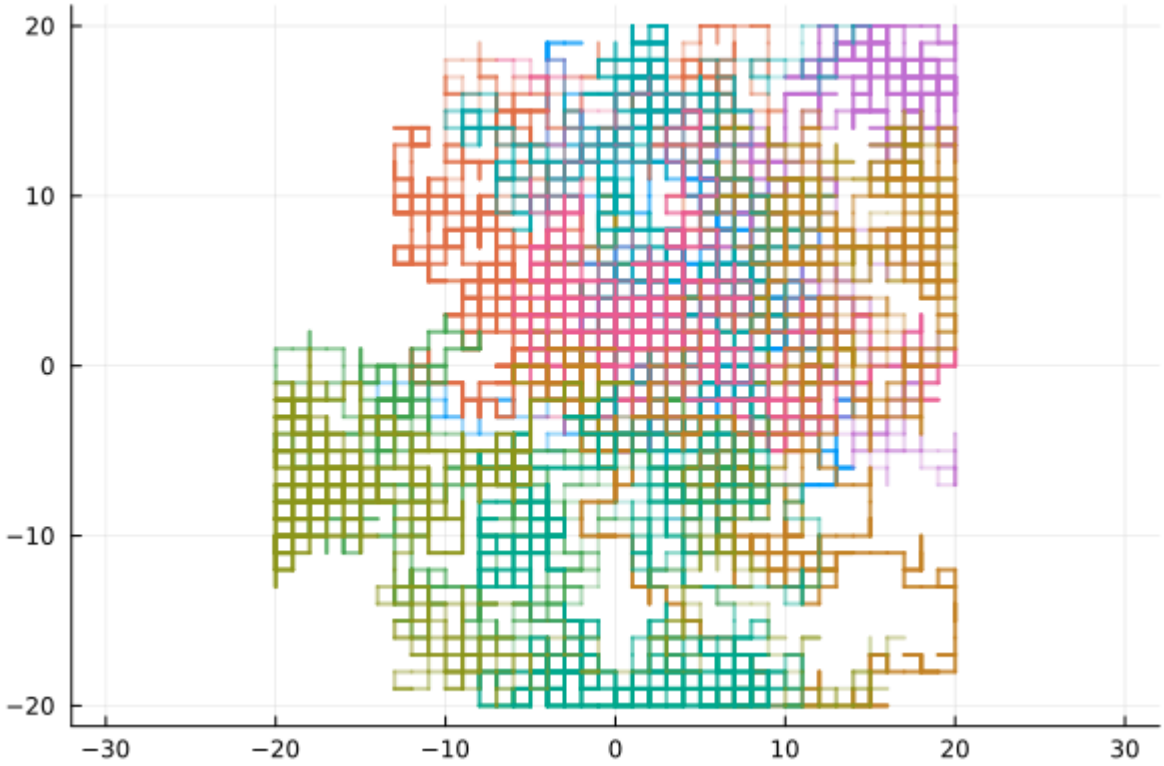
```
collide_boundary(Coordinate(12,4), 10) # uncomment to test
```

Exercise 1.6

👉 Implement a 3-argument method of `trajectory` where the third argument is a size. The trajectory returned should be within the boundary (use `collide_boundary` from above). You can still use `accumulate` with an anonymous function that makes a move and then reflects the resulting coordinate, or use a for loop.

```
trajectory (generic function with 2 methods)
```

```
function trajectory(c::Coordinate, n::Int, L::Number)
    moves=rand(possible_moves,n)
    f=(p::Coordinate,q::Coordinate)->collide_boundary(p+q,L)
    return accumulate(f,moves,init=c)
end
```




```
let
    p = plot(ratio=1)
    for i in 1:10
        long_trajectory = trajectory(origin, 1000,20)
        plot_trajectory!(p, long_trajectory)
    end
        p
end
```

```
bigbreak
```

Exercise 2: *Wanderering Agents*

In this exercise we will create Agents which have a location as well as some infection state information.

Let's define a type `Agent`. `Agent` contains a `position` (of type `Coordinate`), as well as a `status` of type `InfectionStatus` (as in Homework 4).)

(For simplicity we will not use a `num_infected` field, but feel free to do so!)

```
@enum InfectionStatus S I R
```

```
# define agent struct here:
mutable struct Agent
    position::Coordinate
    status::InfectionStatus
end
```

Exercise 2.1

👉 Write a function `initialize` that takes parameters N and L, where N is the number of agents and $2L$ is the side length of the square box where the agents live.

It returns a `Vector` of `N` randomly generated `Agent`s. Their coordinates are randomly sampled in the $[-L, L] \times [-L, L]$ box, and the agents are all susceptible, except one, chosen at random, which is infectious.

```
initialize (generic function with 1 method)
```

```
function initialize(N::Int, L::Int)
    agents = [Agent(Coordinate(rand(-L:L),rand(-L:L)), S) for i in 1:N]
    infected_agent_index = rand(1:N)
    agents[infected_agent_index] = Agent(agents[infected_agent_index].position, I)
    return agents
end
```

```
[Agent(Coordinate(-6, 8), I::InfectionStatus = 1), Agent(Coordinate(-5, 1), S::InfectionStatu
```

```
initialize(3, 10)
```

Got it!

Yay ❤

```
color (generic function with 2 methods)
```

```
# Color based on infection status
color(s::InfectionStatus) = if s == S
    "blue"
elseif s == I
    "red"
else
    "green"
end
```

```
position (generic function with 1 method)
```

```
position(a::Agent) = a.position # uncomment this line
```

```
color (generic function with 2 methods)
```

```
color(a::Agent) = color(a.status) # uncomment this line
```

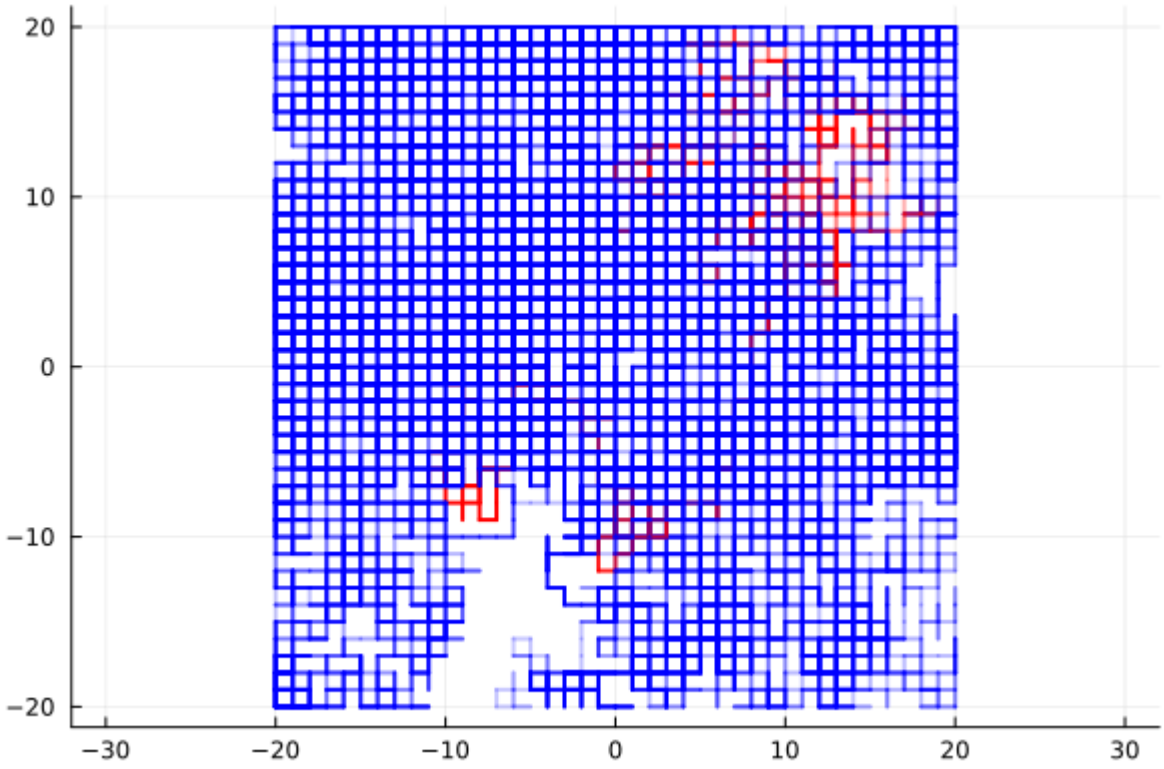
Exercise 2.2

👉 Write a function `visualize` that takes in a collection of agents as argument, and the box size `L`. It should plot a point for each agent at its location, coloured according to its status.

You can use the keyword argument `c=color.(agents)` inside your call to the plotting function make the point colors correspond to the infection statuses. Don't forget to use `ratio=1`.

```
visualize (generic function with 1 method)
```

```
function visualize(agents::Vector, L)
    p = plot(ratio=1)
    for i in 1:length(agents)
        long_trajectory = trajectory(agents[i].position, 1000,20)
        plot_trajectory!(p, long_trajectory,c=color(agents[i].status))
    end
    return p
end
```




```
let
    N = 20
    L = 10
    visualize(initialize(N, L), L) # uncomment this line!
end
```

Exercise 3: Spatial epidemic model – Dynamics

Last week we wrote a function `interact!` that takes two agents, `agent` and `source`, and an infection of type `InfectionRecovery`, which models the interaction between two agent, and possibly modifies `agent` with a new status.

This week, we define a new infection type, `CollisionInfectionRecovery`, and a new method that is the same as last week, except it **only infects `agent` if `agents.position==source.position`**.

```
abstract type AbstractInfection end
```

```
struct CollisionInfectionRecovery <: AbstractInfection
    p_infection::Float64
    p_recovery::Float64
end
```

Write a function `interact!` that takes two `Agent`s and a `CollisionInfectionRecovery`, and:

- If the agents are at the same spot, causes a susceptible agent to communicate the disease from an infectious one with the correct probability.
- if the first agent is infectious, it recovers with some probability

```
interact! (generic function with 1 method)
```

```
function interact!(agent::Agent, source::Agent, infection::CollisionInfectionRecovery)
    if agent.position==source.position && source.status==I && rand()
    <infection.p_infection && agent.status==S
        agent.status=I
    elseif agent.status==I && rand()<infection.p_recovery
        agent.status=R
    end
end
```

Exercise 3.1

Your turn!

👉 Write a function `step!` that takes a vector of `Agent`s, a box size `L` and an `infection`. This that does one step of the dynamics on a vector of agents.

- Choose an Agent `source` at random.
- Move the `source` one step, and use `collide_boundary` to ensure that our agent stays within the box.
- For all *other* agents, call `interact!(other_agent, source, infection)`.
- return the array `agents` again.

```
step! (generic function with 1 method)
```

```
function step!(agents::Vector, L::Number, infection::AbstractInfection)
    source_index=rand(1:length(agents))
    source=agents[source_index]
    source.position=collide_boundary(source.position+rand(possible_moves),L)
    for i in 1:length(agents)
        if i==source_index
            continue
        else
            interact!(agents[i],source,infection)
        end
    end
    return agents
end
```

Exercise 3.2

If we call `step! N` times, then every agent will have made one step, on average. Let's call this one *sweep* of the simulation.

👉 Create a before-and-after plot of $k_{sweeps} = 1000$ sweeps.

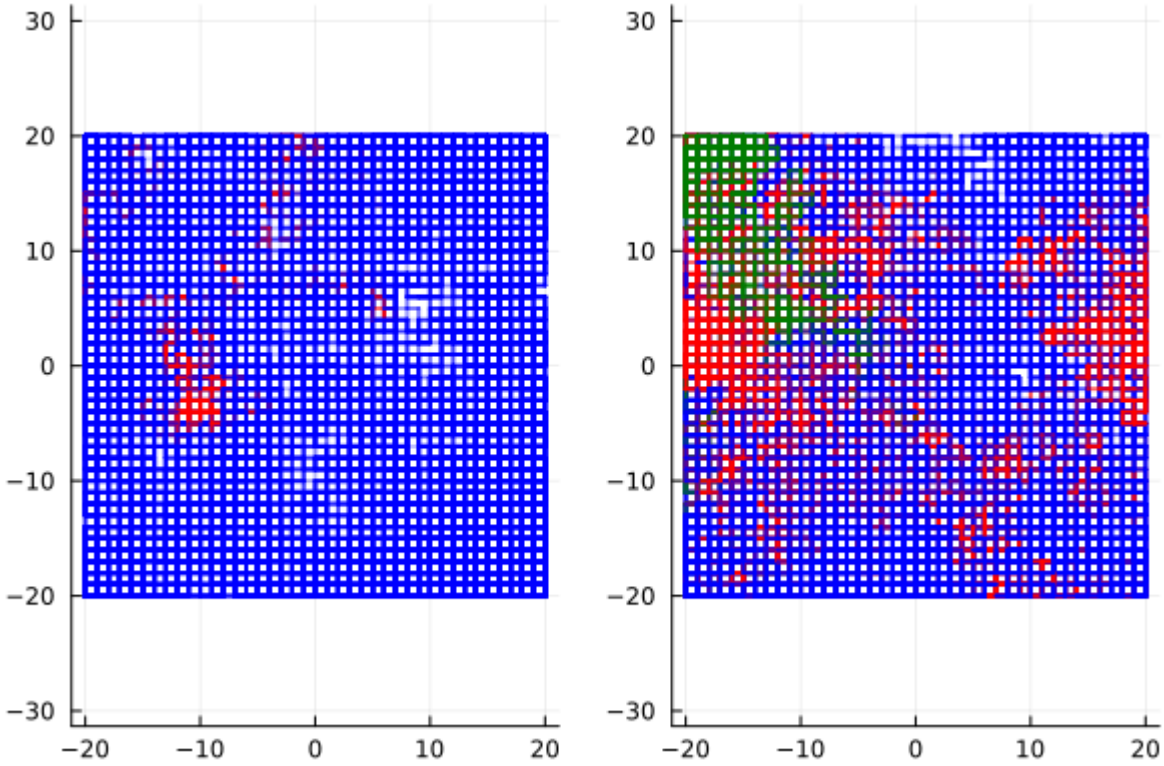
- Initialize a new vector of agents (`N=50`, `L=40`, `infection` is given as `pandemic` below).
- Plot the state using `visualize`, and save the plot as a variable `plot_before`.
- Run `k_sweeps` sweeps.
- Plot the state again, and store as `plot_after`.
- Combine the two plots into a single figure using

```
plot(plot_before, plot_after)
```

```
pandemic =  CollisionInfectionRecovery(0.5, 1.0e-5)
```

```
1 pandemic = CollisionInfectionRecovery(0.5, 0.00001)
```

```
1 @bind k_sweeps Slider(1:10000, default=1000)
```




```
let
    N = 50
    L = 40
    agents=initialize(N,L)
    plot_before = visualize(agents,L)
    for i in 1:k_sweeps
        for j in 1:N
            agents=step!(agents,L,pandemic)
        end
    end
    plot_after = visualize(agents,L)

    plot(plot_before, plot_after)
end
```

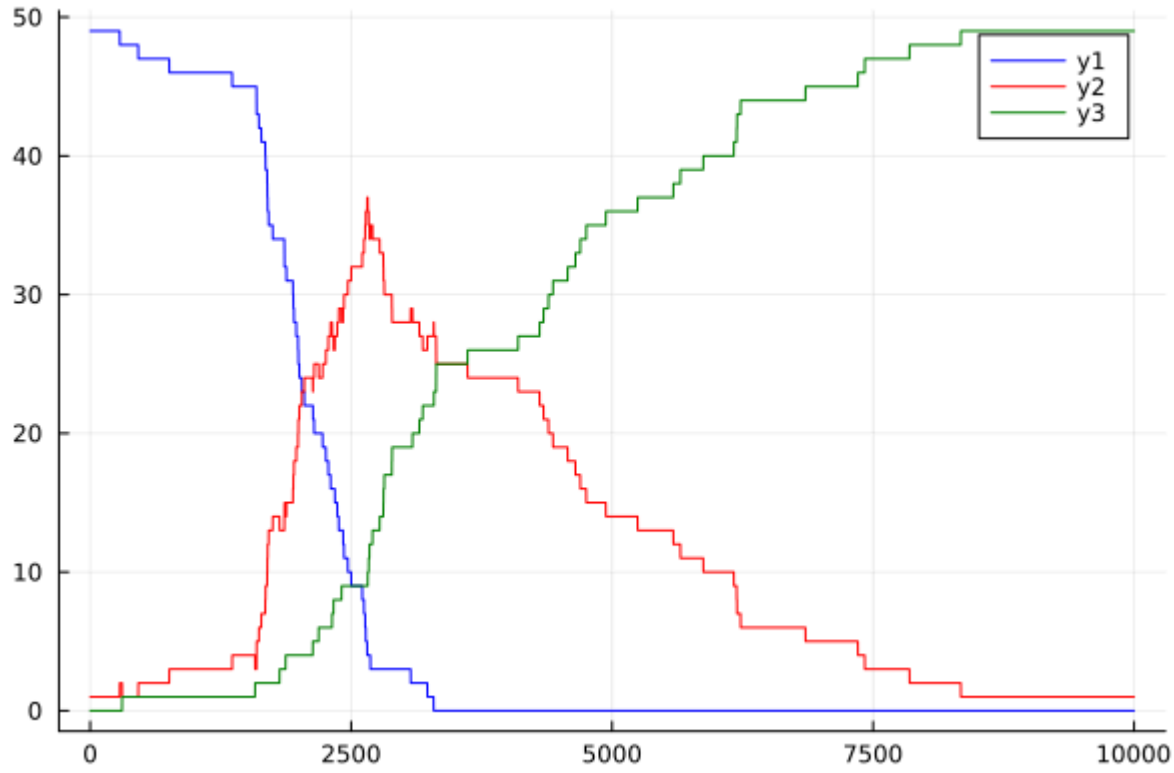
Exercise 3.3

Every time that you move the slider, a completely new simulation is created an run. This makes it hard to view the progress of a single simulation over time. So in this exercise, we we look at a single simulation, and plot the S, I and R curves.

👉 Plot the SIR curves of a single simulation, with the same parameters as in the previous exercise. Use `k_sweep_max = 10000` as the total number of sweeps.

```
k_sweep_max = 10000
```

```
k_sweep_max = 10000
```




```
let
    N = 50
    L = 30

    agents = initialize(N, L)
    curve = zeros(k_sweep_max, 3)
    for i in 1:k_sweep_max
        s = 0
        j = 0
        r = 0
        for agent in agents
            if agent.status == S
                s += 1
            elseif agent.status == I
                j += 1
            else
                r += 1
            end
        end
        curve[i,1] = s
        curve[i,2] = j
        curve[i,3] = r
        for k in 1:N
            agents=step!(agents,L,pandemic)
        end
    end
    time = 1:k_sweep_max
    s = curve[:,1]
    j = curve[:,2]
    r = curve[:,3]
    p = plot()
    plot!(time, s, color=:blue)
    plot!(time, j, color=:red)
    plot!(time, r, color=:green)
    p
```

```
36 end
```

> **Hint**
>
> After every sweep, count the values S, I and R and push! them to 3 arrays.

Exercise 3.4 (optional)

Let's make our plot come alive! There are two options to make our visualization dynamic:

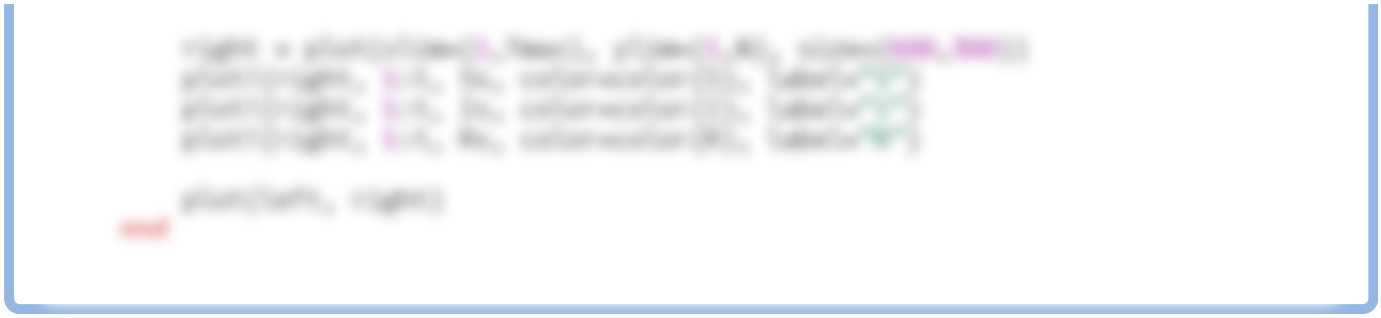
👉 1️⃣ Precompute one simulation run and save its intermediate states using `deepcopy`. You can then write an interactive visualization that shows both the state at time t (using `visualize`) and the history of S, I and R from time 0 up to time t. t is controlled by a slider.

👉 2️⃣ Use `@gif` from Plots.jl to turn a sequence of plots into an animation. Be careful to skip about 50 sweeps between each animation frame, otherwise the GIF becomes too large.

This an optional exercise, and our solution to 2️⃣ is given below.

```
let
    N = 50
    L = 40

    x = initialize(N, L)

    # initialize to empty arrays
    Ss, Is, Rs = Int[], Int[], Int[]

    Tmax = 200

    @gif for t in 1:Tmax
        for i in 1:50N
            step!(x, L, pandemic)
        end

        #... track S, I, R in Ss Is and Rs
        s_count = count(agent -> agent.status == S, x)
        i_count = count(agent -> agent.status == I, x)
        r_count = count(agent -> agent.status == R, x)

    # Add the new counts to our data arrays
        push!(Ss, s_count)
        push!(Is, i_count)
        push!(Rs, r_count)


        left = visualize(x, L)

        right = plot(xlim=(1,Tmax), ylim=(1,N), size=(600,300))
        plot!(right, 1:t, Ss, color=color(S), label="S")
        plot!(right, 1:t, Is, color=color(I), label="I")
        plot!(right, 1:t, Rs, color=color(R), label="R")

        plot(left, right)
    end
end
```

Hint



Exercise 3.5

👉 Using $L = 20$ and $N = 100$, experiment with the infection and recovery probabilities until you find an epidemic outbreak. (Take the recovery probability quite small.) Modify the two infections below to match your observations.

```
causes_outbreak =   CollisionInfectionRecovery(0.6, 0.00013)
```

```
1 causes_outbreak = CollisionInfectionRecovery(0.6, 0.00013)
```

```
does_not_cause_outbreak =   CollisionInfectionRecovery(0.5, 0.001)
```

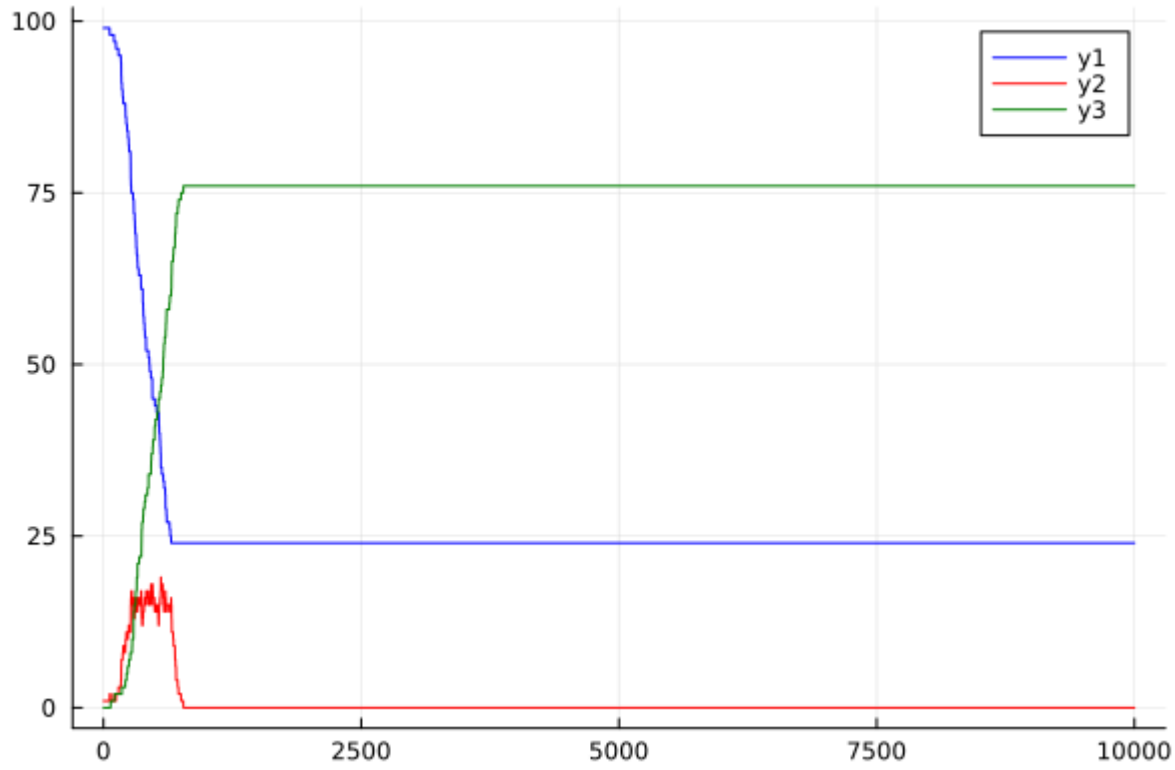
```
1 does_not_cause_outbreak = CollisionInfectionRecovery(0.5, 0.001)
```

0.5

```
1 @bind inf_prob Slider(0:0.01:1, default=0.5,show_value=true)
```

0.0001

```
1 @bind rec_prob Slider(0:0.00001:0.001, default=0.0001,show_value=true)
```

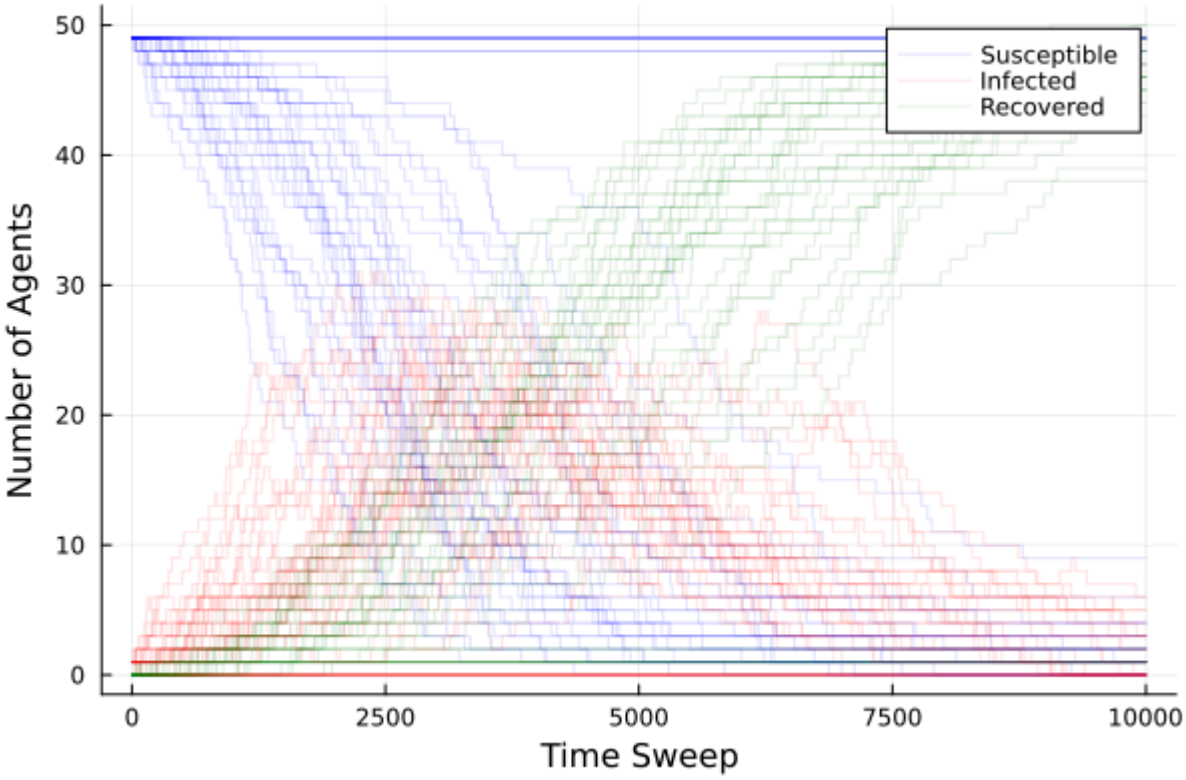



```
let
    L=20
    N=100
    my_pandemic=CollisionInfectionRecovery(inf_prob,rec_prob)

    agents = initialize(N, L)
    curve = zeros(k_sweep_max, 3)
    for i in 1:k_sweep_max
        s = 0
        j = 0
        r = 0
        for agent in agents
            if agent.status == S
                s += 1
            elseif agent.status == I
                j += 1
            else
                r += 1
            end
        end
        curve[i,1] = s
        curve[i,2] = j
        curve[i,3] = r
        for k in 1:N
            agents=step!(agents,L,my_pandemic)
        end
    end
    time = 1:k_sweep_max
    s = curve[:,1]
    j = curve[:,2]
    r = curve[:,3]
    p = plot()
    plot!(time, s, color=:blue)
    plot!(time, j, color=:red)
    plot!(time, r, color=:green)
```

```
36         p
37     end
```

Exercise 3.6

👉 With the parameters of Exercise 3.2, run 50 simulations. Plot S, I and R as a function of time for each of them (with transparency!). This should look qualitatively similar to what you saw in the previous homework. You probably need different `p_infection` and `p_recovery` values from last week. Why?




```julia
let
    N=50
    L=40
    p = plot(
        xlabel="Time Sweep",
        ylabel="Number of Agents"
    )
    for sim in 1:50
        agents = initialize(N, L)
        curve = zeros(Int, k_sweep_max, 3)
        for i in 1:k_sweep_max
            s_count = count(a -> a.status == S, agents)
            i_count = count(a -> a.status == I, agents)
            r_count = N - s_count - i_count
            curve[i, :] = [s_count, i_count, r_count]
            for k in 1:N
                agents = step!(agents, L, pandemic)
            end
        end
        time = 1:k_sweep_max
        s_label = (sim == 1) ? "Susceptible" : ""
        i_label = (sim == 1) ? "Infected" : ""
        r_label = (sim == 1) ? "Recovered" : ""

        plot!(p, time, curve[:, 1], color=:blue, linealpha=0.1, label=s_label)
        plot!(p, time, curve[:, 2], color=:red, linealpha=0.1, label=i_label)
        plot!(p, time, curve[:, 3], color=:green, linealpha=0.1, label=r_label)
    end
    p
end
```

need_different_parameters_because =
interaction function is different from the previous homework.

```
need_different_parameters_because = md"""
interaction function is different from the previous homework.
"""
```

Exercise 4: (Optional) *Effect of socialization*

In this exercise we'll modify the simple mixing model. Instead of a constant mixing probability, i.e. a constant probability that any pair of people interact on a given day, we will have a variable probability associated with each agent, modelling the fact that some people are more or less social or contagious than others.

Exercise 4.1

We create a new agent type `SocialAgent` with fields `position`, `status`, `num_infected`, and `social_score`. The attribute `social_score` represents an agent's probability of interacting with any other agent in the population.

```
mutable struct SocialAgent <: AbstractAgent
    position::Coordinate
    status::InfectionStatus
    num_infected::Int
    social_score::Float64
end
```

define the `position` and `color` methods for `SocialAgent` as we did for `Agent`. This will allow the `visualize` function to work. on both kinds of Agents

```
color (generic function with 3 methods)
```

```
begin
    position(a::SocialAgent) = a.position
    color(a::SocialAgent) = color(a.status)
end
```

👉 Create a function `initialize_social` that takes `N` and `L`, and creates N agents within a 2L x 2L box, with `social_score`s chosen from 10 equally-spaced between 0.1 and 0.5. (see LinRange)

```
initialize_social (generic function with 1 method)
```

```
function initialize_social(N, L)
    agents = [SocialAgent(Coordinate(rand(-L:L),rand(-L:L)),
    S,0,rand(LinRange(0.1,0.5,10))) for i in 1:N]
    infected_agent_index = rand(1:N)
    agents[infected_agent_index] = SocialAgent(agents[infected_agent_index].position,
    I,0,agents[infected_agent_index].social_score)
    return agents
end
```

Now that we have 2 agent types

1. let's create an AbstractAgent type
2. Go back in the notebook and make the agent types a subtype of AbstractAgent.

```
abstract type AbstractAgent end
```

Exercise 4.2

Not all two agents who end up in the same grid point may actually interact in an infectious way – they may just be passing by and do not create enough exposure for communicating the disease.

👉 Write a new `interact!` method on `SocialAgent` which adds together the social_scores for two agents and uses that as the probability that they interact in a risky way. Only if they interact in a risky way, the infection is transmitted with the usual probability.

```
interact! (generic function with 2 methods)
```

```
function interact!(agent::SocialAgent, source::SocialAgent,
infection::CollisionInfectionRecovery)

    # your code here
    if agent.position==source.position && source.status==I && rand()
    <infection.p_infection && rand()<agent.social_score+source.social_score &&
    agent.status==S
        agent.status=I
        source.num_infected+=1
    elseif agent.status==I && rand()<infection.p_recovery
        agent.status=R
    end

end
```

Make sure `step!`, `position`, `color`, work on the type `SocialAgent`. If `step!` takes an untyped first argument, it should work for both Agent and SocialAgent types without any changes. We actually only need to specialize `interact!` on SocialAgent.

Exercise 4.3

👉 Plot the SIR curves of the resulting simulation.

N = 50; L = 40; number of steps = 200

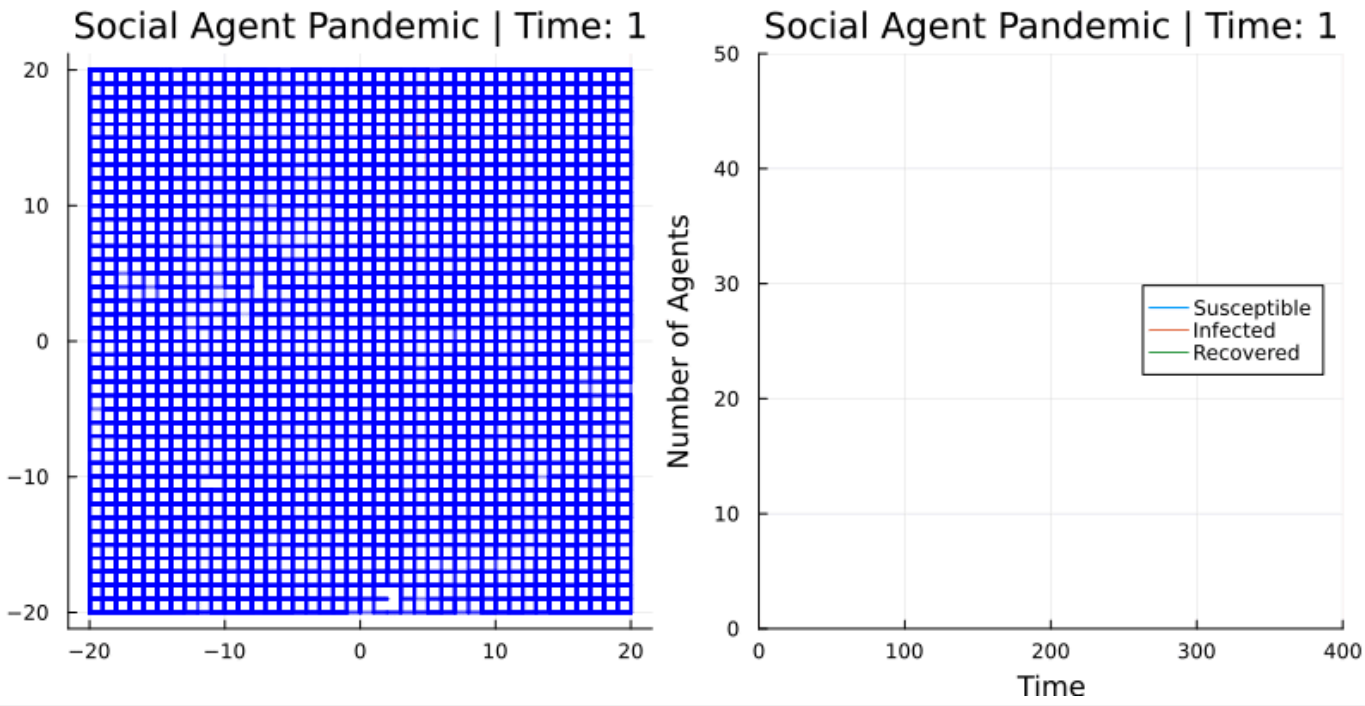
In each step call `step!` 50N times.

```
visualize_sir (generic function with 1 method)
```

```
function visualize_sir(Ss, Is, Rs, Tmax, N)
    time_steps = 1:length(Ss)
    return plot(time_steps, [Ss Is Rs],
        label=["Susceptible" "Infected" "Recovered"],
        xlabel="Time",
        ylabel="Number of Agents",
        title="SIR Model",
        xlims=(0, Tmax),
        ylims=(0, N),
        legend=:right
    )
end
```

```
[SocialAgent(Coordinate(18, 38), I::InfectionStatus = 1, 0, 0.1), SocialAgent(Coordinate(2, -9
```

```
begin
    N = 50
    L = 40
    Ss, Is, Rs = Int[], Int[], Int[]
    global social_agents = initialize_social(N, L)
end
```

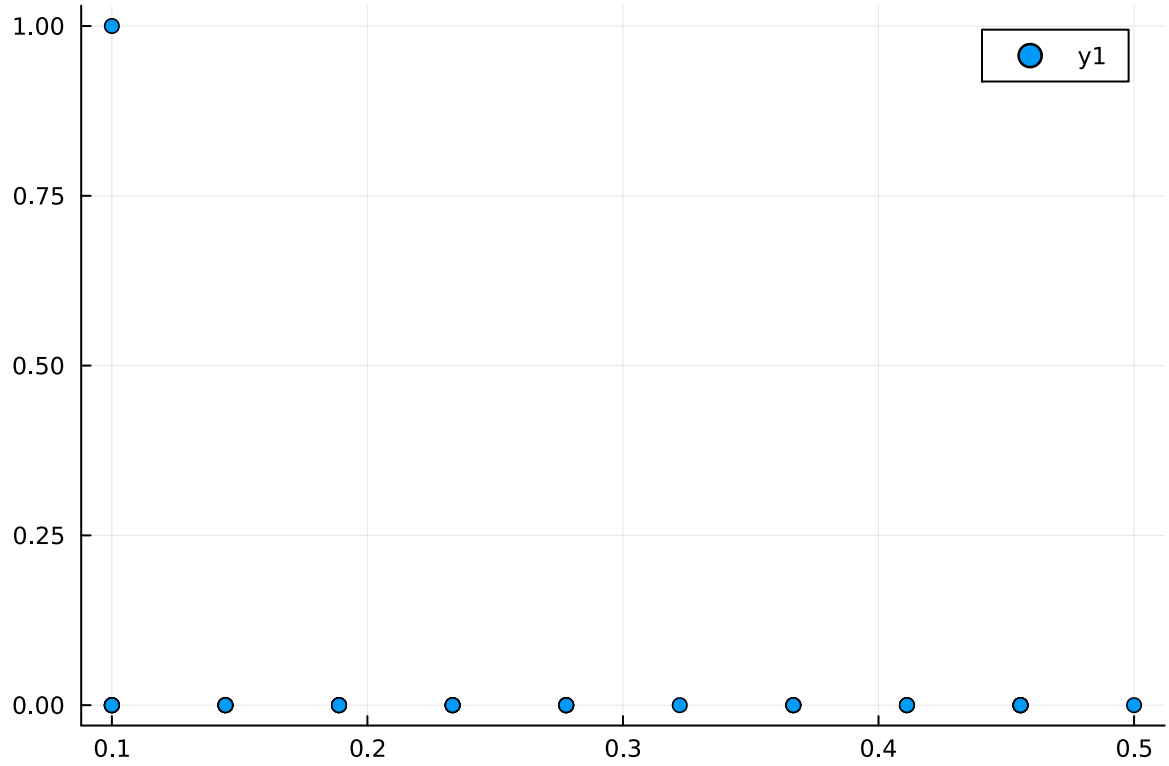



```
begin
    Tmax = 400

    @gif for t in 1:Tmax
            for j in 1:5*N
                social_agents=step!(social_agents,L,pandemic)
            end
        s = count(agent.status == S for agent in social_agents)
        i = count(agent.status == I for agent in social_agents)
        r = count(agent.status == R for agent in social_agents)

        push!(Ss, s)
        push!(Is, i)
        push!(Rs, r)
        p_agents = visualize(social_agents, L)
        p_sir = visualize_sir(Ss, Is, Rs, Tmax, N)
        plot(p_agents, p_sir, layout=(1,2), size=(800, 400), title="Social Agent
        Pandemic | Time: $t")
    end
end
```

```
Saved animation to
fn: "C:\\Users\\Dell\\AppData\\Local\\Temp\\jl_ZaZmVaZuji.gif"
```

Exercise 4.4

👉 Make a scatter plot showing each agent's `social_score` on one axis, and the `num_infected` from the simulation in the other axis. Run this simulation several times and comment on the results.

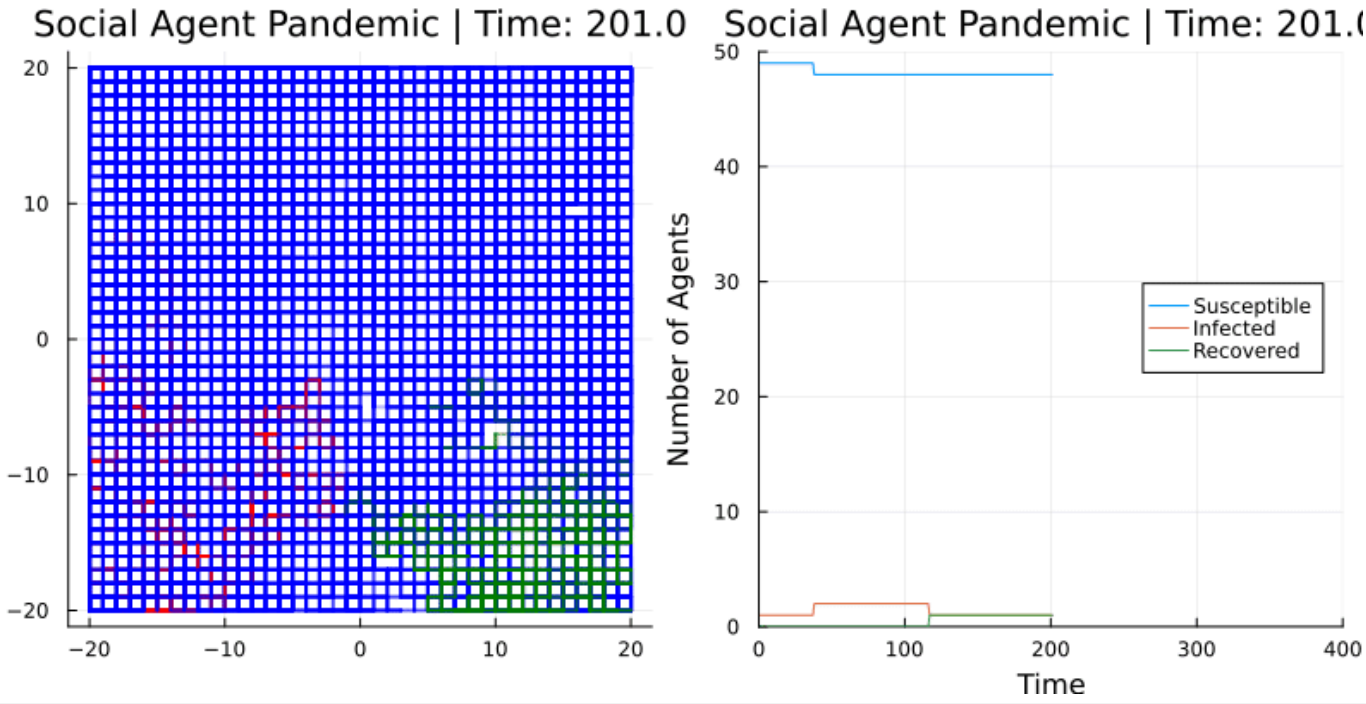



```
begin
    social_array=zeros(length(social_agents))
    inf_array=zeros(length(social_agents))
    for i in 1:length(social_agents)
        social_array[i]=social_agents[i].social_score
        inf_array[i]=social_agents[i].num_infected
    end
    scatter(social_array,inf_array)
end
```

👉 Run a simulation for 100 steps, and then apply a "lockdown" where every agent's social score gets multiplied by 0.25, and then run a second simulation which runs on that same population from there. What do you notice? How does changing this factor form 0.25 to other numbers affect things?

```
[SocialAgent(Coordinate(-5, -36), S::InfectionStatus = 0, 0, 0.188889), SocialAgent(Coordinate
```

```
begin
    N_lcd = 50
    L_lcd = 40
    Ss_lcd, Is_lcd, Rs_lcd = Int[], Int[], Int[]
    global social_agents_lcd = initialize_social(N_lcd, L_lcd)
end
```




```
begin
    Tmax_lcd = 400

    @gif for t in 1:Tmax_lcd/2
            for j in 1:5*N_lcd
                social_agents_lcd=step!(social_agents_lcd,L,pandemic)
            end
        s_lcd = count(agent.status == S for agent in social_agents_lcd)
        i_lcd = count(agent.status == I for agent in social_agents_lcd)
        r_lcd = count(agent.status == R for agent in social_agents_lcd)

        push!(Ss_lcd, s_lcd)
        push!(Is_lcd, i_lcd)
        push!(Rs_lcd, r_lcd)
        p_agents_lcd = visualize(social_agents_lcd, L_lcd)
        p_sir_lcd = visualize_sir(Ss_lcd, Is_lcd, Rs_lcd, Tmax_lcd, N_lcd)
        plot(p_agents_lcd, p_sir_lcd, layout=(1,2), size=(800, 400), title="Social
        Agent Pandemic | Time: $t")
    end
    @gif for t in Tmax_lcd/2 + 1:Tmax_lcd
            for j in 1:5*N_lcd
                social_agents_lcd=step!(social_agents_lcd,L,pandemic)
            end
        s_lcd = count(agent.status == S for agent in social_agents_lcd)
        i_lcd = count(agent.status == I for agent in social_agents_lcd)
        r_lcd = count(agent.status == R for agent in social_agents_lcd)

        push!(Ss_lcd, s_lcd)
        push!(Is_lcd, i_lcd)
        push!(Rs_lcd, r_lcd)
        p_agents_lcd = visualize(social_agents_lcd, L_lcd)
        p_sir_lcd = visualize_sir(Ss_lcd, Is_lcd, Rs_lcd, Tmax_lcd, N_lcd)
        plot(p_agents_lcd, p_sir_lcd, layout=(1,2), size=(800, 400), title="Social
        Agent Pandemic | Time: $t")
    end
end
```

```
Saved animation to
```

```
fn: "C:\\Users\\Dell\\AppData\\Local\\Temp\\jl_jaVSrEfI1W.gif"

Saved animation to
fn: "C:\\Users\\Dell\\AppData\\Local\\Temp\\jl_zSlgREnoN6.gif"
```

Exercise 5: (Optional) *Effect of distancing*

We can use a variant of the above model to investigate the effect of the mis-named "social distancing" (we want people to be *socially* close, but *physically* distant).

In this variant, we separate out the two effects "infection" and "movement": an infected agent chooses a neighbouring site, and if it finds a susceptible there then it infects it with probability p_I. For simplicity we can ignore recovery.

Separately, an agent chooses a neighbouring site to move to, and moves there with probability p_M if the site is vacant. (Otherwise it stays where it is.)

When $p_M = 0$, the agents cannot move, and hence are completely quarantined in their original locations.

👉 How does the disease spread in this case?

```
begin
    mutable struct PhysicalDistancingAgent <: AbstractAgent
        position::Coordinate
        status::InfectionStatus
    end
end
```

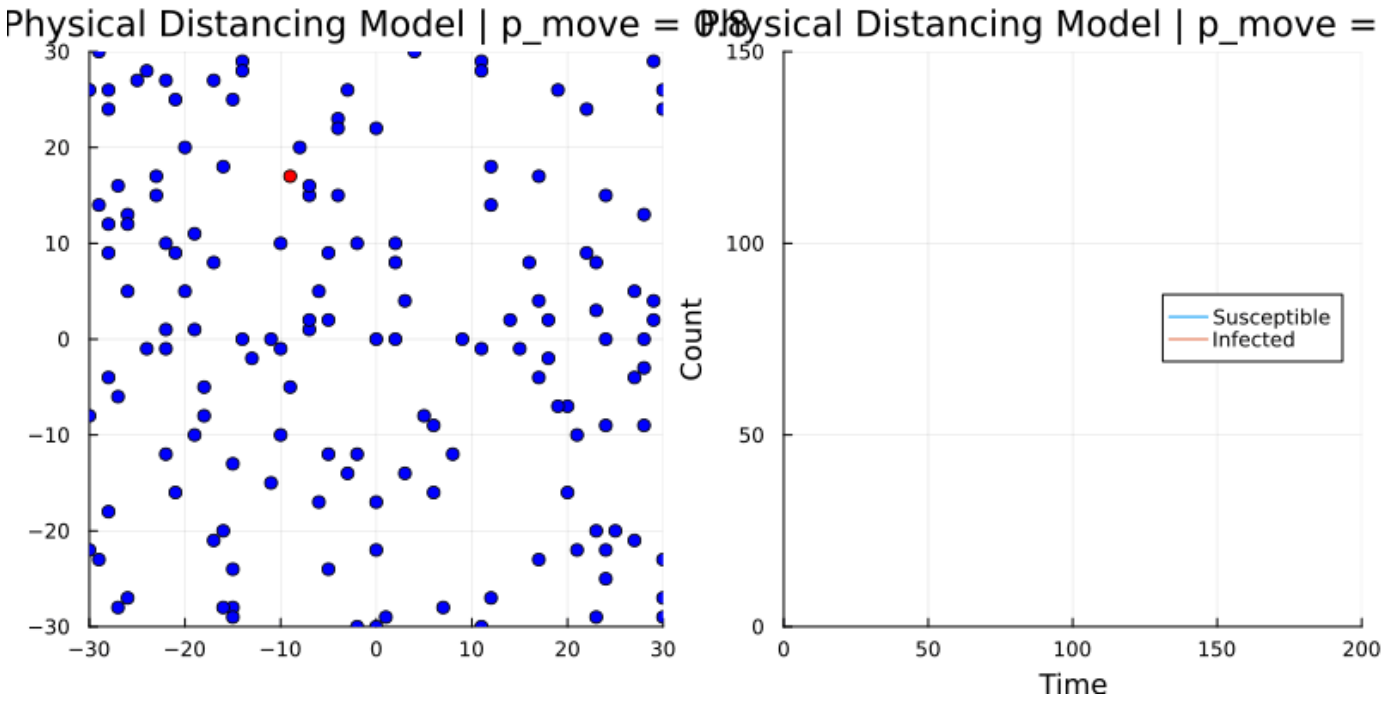
```
initialize_physical_distancing (generic function with 1 method)
```

```
begin
    function get_neighbours(pos::Coordinate, L::Int)
        neighbours = Coordinate[]
        for dx in -1:1, dy in -1:1
            if dx == 0 && dy == 0
                continue
            end
            new_x = mod(pos.x - (-L) + dx, 2L + 1) + (-L)
            new_y = mod(pos.y - (-L) + dy, 2L + 1) + (-L)
            push!(neighbours, Coordinate(new_x, new_y))
        end
        return neighbours
    end

    function initialize_physical_distancing(N::Int, L::Int)
        all_sites = [Coordinate(x, y) for x in -L:L for y in -L:L]
        if N > length(all_sites)
            error("N cannot be larger than the number of available grid sites.")
        end
        agent_positions = Set(rand(all_sites, N))

        agents = [PhysicalDistancingAgent(pos, S) for pos in agent_positions]
        if !isempty(agents)
            rand(agents).status = I
        end

        return agents
    end
end
```

```
step_physical_distancing! (generic function with 1 method)
```

```
function step_physical_distancing!(agents::Vector{PhysicalDistancingAgent}, L::Int,
p_infection::Float64, p_move::Float64)
    agent_positions = Dict(agent.position => agent for agent in agents)
    for i in 1:length(agents)
        agent = agents[i]
        if agent.status == I
            neighbours = get_neighbours(agent.position, L)
            for neighbour_pos in neighbours
                if haskey(agent_positions, neighbour_pos)
                    target_agent = agent_positions[neighbour_pos]
                    if target_agent.status == S && rand() < p_infection
                        target_agent.status = I
                    end
                end
            end
        end
        if rand() < p_move
            potential_move_pos = rand(get_neighbours(agent.position, L))
            if !haskey(agent_positions, potential_move_pos)
                delete!(agent_positions, agent.position)
                agent_positions[potential_move_pos] = agent
                agent.position = potential_move_pos
            end
        end
    end
end
```

👉 Run the dynamics repeatedly, and plot the sites which become infected.




```
let
    N = 150
    L = 30
    Tmax = 200
    p_infection = 0.1
    p_move = 0.8
    agents = initialize_physical_distancing(N, L)
    Ss, Is = Int[], Int[]
    function visualize_grid(agents, L)
        colors = map(a -> a.status == S ? :blue : :red, agents)
        positions_x = [a.position.x for a in agents]
        positions_y = [a.position.y for a in agents]

        scatter(positions_x, positions_y, markercolor=colors, label="", title="Agent
        Positions", xlims=(-L, L), ylims=(-L, L), aspect_ratio=:equal, markersize=4,
        background_color=:whitesmoke)
    end

    function visualize_si_curve(Ss, Is, Tmax, N)
        plot(1:length(Ss), [Ss Is], label=["Susceptible" "Infected"], xlabel="Time",
        ylabel="Count", title="SI Curve", xlims=(0, Tmax), ylims=(0, N),
        legend=:right)
    end
    @gif for t in 1:Tmax
        step_physical_distancing!(agents, L, p_infection, p_move)

        s_count = count(a -> a.status == S, agents)
        i_count = N - s_count
        push!(Ss, s_count); push!(Is, i_count)

        p1 = visualize_grid(agents, L)
        p2 = visualize_si_curve(Ss, Is, Tmax, N)
        plot(p1, p2, layout=(1,2), size=(800, 400), title="Physical Distancing Model
        | p_move = $p_move")
    end
end
```

```
Saved animation to
```

```
fn: "C:\\Users\\Dell\\AppData\\Local\\Temp\\jl_3UheX2p2ni.gif"
```

👉 How does this change as you increase the *density* $\rho = N/(L^2)$ of agents? Start with a small density.

This is basically the **site percolation** model.

When we increase p_M, we allow some local motion via random walks.

```
0.03125
```

```
begin
    p_infection = 0.1
    p_move = 0.5
    total_sites = (2*L + 1)^2
    low_density = 0.01
    high_density = N/L^2
end
```

```
(
    Ss =  [48, 48, 48, 48, 48, 48, 48, 48, 48,  more ,46]
    Is =  [2, 2, 2, 2, 2, 2, 2, 2, 2,  more ,4]
)
```

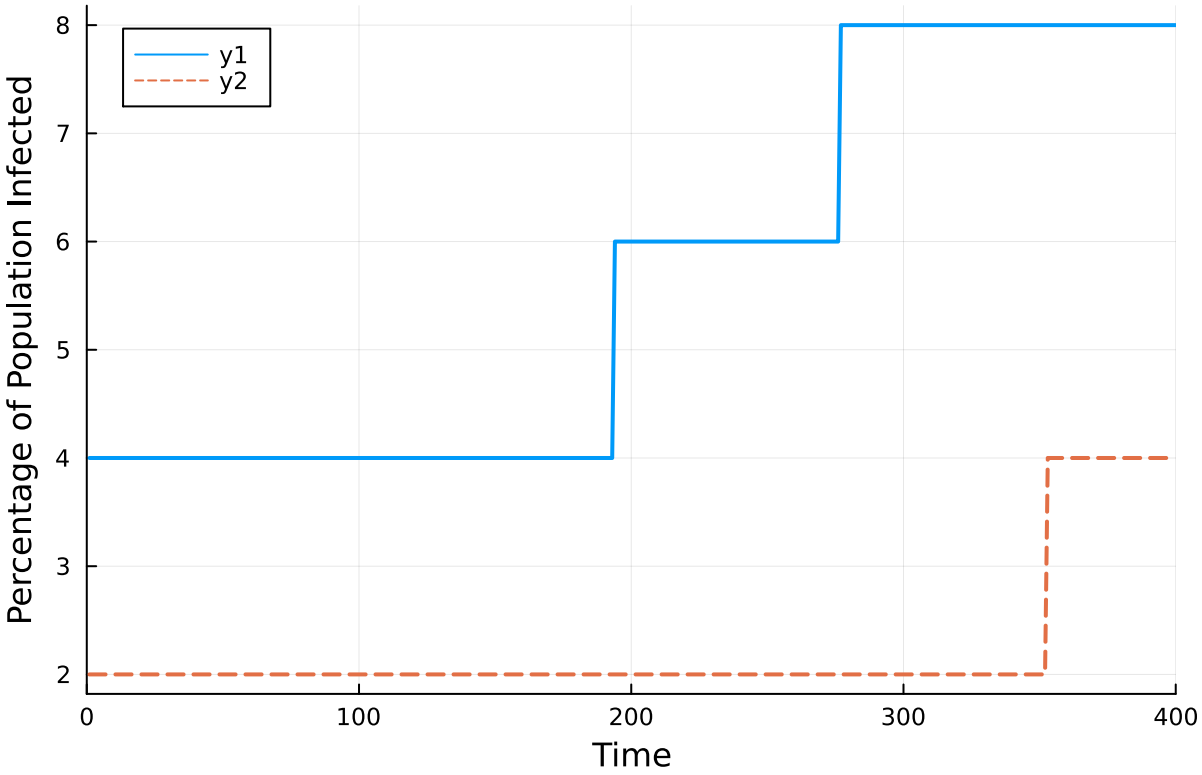
```
let
    agents_low = initialize_physical_distancing(N, L)
    Ss_low, Is_low,Rs_low = Int[], Int[],Int[]
    @gif for t in 1:Tmax
        step_physical_distancing!(agents_low, L, p_infection, p_move)

        s_count = count(a -> a.status == S, agents_low)
        i_count = N - s_count
        r_count = 0
        push!(Ss_low, s_count); push!(Is_low, i_count);push!(Rs_low, r_count)
        p1 = visualize(agents_low, L)
        p2 = visualize_sir(Ss_low, Is_low,Rs_low, Tmax, N)
        plot(p1, p2, layout=(1,2), size=(800, 400), title="Low Density (10%) | p_move
        = $p_move")
    end
    global results_low = (Ss=Ss_low, Is=Is_low)
end
```

```
Saved animation to
fn: "C:\\Users\\Dell\\AppData\\Local\\Temp\\jl_PUv2joDMJ8.gif"
```

```
(Ss = [49, 49, 49, 49, 49, 49, 49, 49, 49,    more ,48], Is = [1, 1, 1, 1, 1, 1, 1, 1, 1,    more
```

```
let
    agents_high = initialize_physical_distancing(N, L)
    Ss_high, Is_high,Rs_high = Int[], Int[], Int[]
    @gif for t in 1:Tmax
        step_physical_distancing!(agents_high, L, p_infection, p_move)
        s_count = count(a -> a.status == S, agents_high)
        i_count = N - s_count
        r_count = 0
        push!(Ss_high, s_count); push!(Is_high, i_count);push!(Rs_high,r_count)

        p1 = visualize(agents_high, L)
        p2 = visualize_sir(Ss_high, Is_high,Rs_high, Tmax, N)
        plot(p1, p2, layout=(1,2), size=(800, 400), title="High Density (40%) |
        p_move = $p_move")
    end
    global results_high = (Ss=Ss_high, Is=Is_high)
end
```

```
Saved animation to
fn: "C:\\Users\\Dell\\AppData\\Local\\Temp\\jl_Js32J2JklA.gif"
```




```
begin
    comparison_plot = plot(
        xlabel="Time",
        ylabel="Percentage of Population Infected",
        legend=:topleft,
        xlims=(0, Tmax)
    )
    time_steps_low = 1:length(results_low.Is)
    infected_percent_low = (results_low.Is ./ N) .* 100
    plot!(comparison_plot, time_steps_low, infected_percent_low,
        linewidth=2
    )
    time_steps_high = 1:length(results_high.Is)
    infected_percent_high = (results_high.Is ./ N) .* 100
    plot!(comparison_plot, time_steps_high, infected_percent_high,
        linewidth=2,
        linestyle=:dash
    )
    comparison_plot
end
```

👉 Investigate how this leaky quarantine affects the infection dynamics with different densities.

```
Enter cell code...
```

Error message

```
UndefVarError: `student` not defined in `Main.var"workspace#34"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
if student.name == "Jazzy Doe"
    md"""
    !!! danger "Before you submit"
        Remember to fill in your **name** and **Kerberos ID** at the top of this notebook.
    """
end
```

Reading hidden

Function library

Just some helper functions used in the notebook.

```
hint (generic function with 1 method)
```

```
almost (generic function with 1 method)
```

```
still_missing (generic function with 2 methods)
```

```
keep_working (generic function with 2 methods)
```

```
yays =
  [Fantastic!, Splendid!, Great!, Yay ♥, Great! 🎉, Well done!, Keep it up!, Good job!, Awesome!,
```

```
correct (generic function with 2 methods)
```

```
not_defined (generic function with 1 method)
```